

HITMAN CLOUDFLUX - Infrastructure Architecture & Troubleshooting Master Guide

Complete Reference: Connections, Zero-Key Security, IAM Roles & Resolution Journey

AWS Account: 327425719754

Region: ap-south-1 (Mumbai)

Repository: sumit-8125/GITOPS-PROJECT-45

Branch: main

1. Architecture & Component Connections (Kaun Kisse Kaise Connected Hai)

1. Network Module (VPC & Subnets)

VPC: 10.0.0.0/16 across 2 Availability Zones.

Subnets: 2 Public (with Internet Gateway), 2 Private App (with Single NAT Gateway), and 2 Dedicated Database Subnets.

Connection: EKS nodes private subnets me run hote hain taaki public exposure na ho. DB subnets completely isolated hain.

2. EKS Cluster (Kubernetes v1.31)

Cluster: hitman-cloudflux-prod .

Nodes: Managed Node Group with c7i-flex.large compute instances.

Addons: CoreDNS, kube-proxy, VPC-CNI, and EBS CSI Controller (IRSA) for persistent volume storage.

3. RDS Database (PostgreSQL 16)

Instance: db.t4g.micro (Free-tier friendly).

Password: Terraform random_password generates 32-character high-entropy secret.

Connection: Security group allows inbound port 5432 strictly from EKS Worker Nodes SG.

4. AWS Secrets Manager

Secret: hitman-cloudflux/prod/backend .

Connection: Stores RDS database URL and runtime credentials. Kubernetes **External Secrets Operator (ESO)** reads from here to create native K8s secrets.

5. Amazon ECR Repositories

Repos: hitman-cloudflux/frontend and hitman-cloudflux/backend .

Connection: GitHub Actions builds container images and pushes them directly using OIDC credentials.

6. S3 Remote State Backend

Bucket: gitops-project-45-bucket .

State Key: hitman-cloudflux/prod/terraform.tfstate in ap-south-1 .

Connection: Centrally stores and locks Terraform state for team collaboration and CI/CD.

2. Zero-Keys Identity & Access Management (Roles Kyu Banaye)

Zero-Static-Keys Philosophy: Repositories me static AWS_ACCESS_KEY_ID aur AWS_SECRET_ACCESS_KEY save karna security risk aur hardcoding hai. Humne AWS OIDC (OpenID Connect) implement kiya hai jisse GitHub Actions runner AWS STS se temporary 1-hour session token receive karta hai bina kisi permanent password ke.

Role Name	Type & Principal	Purpose / Kyu Banaya
github-actions-bootstrap-role	Web Identity (OIDC) Federated via GitHub Actions	GitHub Actions CI/CD runner ko Terraform Plan aur Apply chalane ki permissions deta hai (VPC, EKS, RDS, S3 create karne ke liye).
hitman-cloudflux-prod-github-frontend-ecr	Web Identity (OIDC) Federated via GitHub Actions	Frontend CI pipeline ko Frontend ECR repository me Docker image push karne ki permission deta hai.
hitman-cloudflux-prod-github-backend-ecr	Web Identity (OIDC) Federated via GitHub Actions	Backend CI pipeline ko Backend ECR repository me Docker image push karne ki permission deta hai.
hitman-cloudflux-prod-ebs-csi-controller	IRSA (EKS Pod Identity) K8s ServiceAccount: ebs-csi-controller-sa	EKS ke andar chalne wale EBS CSI driver pod ko AWS EBS volumes create/attach/delete karne ki permission deta hai.
hitman-cloudflux-prod-aws-load-balancer-controller	IRSA (EKS Pod Identity) K8s ServiceAccount: aws-load-balancer-controller	Kubernetes Ingress ko AWS Application Load Balancer (ALB) automatically create aur configure karne ki permission deta hai.
hitman-cloudflux-prod-external-secrets	IRSA (EKS Pod Identity) K8s ServiceAccount: external-secrets	Kubernetes pod ko AWS Secrets Manager se backend secrets read karke K8s Secret me sync karne deta hai.

3. Troubleshooting Journey (Pehle Kya Dikkat Thi → Ab Kya Kiya Toh Solve Hua)

#	Pehle Kya Dikkat Thi (Issue)	Asli Wajah (Root Cause)	Ab Kya Kiya (Solution)
1	Local PowerShell par <code>terraform: command not found</code>	Host machine par Terraform CLI binary installed nahi thi.	Pure execution ko GitHub Actions CI/CD automate kiya via <code>workflow_dispatch</code> .
2	Hardcoded AWS Keys ka security risk	GitHub secrets me static access keys save karne se leak hone aur expire hone ka darr rehta hai.	AWS IAM me OIDC Identity Provider + Trust Role banaya (Zero Static Credentials).
3	EKS Version <code>1.33</code> deployment error	AWS EKS abhi 1.33 version support nahi karta (invalid version).	<code>variables.tf</code> aur <code>terraform.tfvars</code> me stable 1.31 configure kiya.
4	Headless CI me prompts par pipeline rukna	<code>github_org</code> aur repository variables me default value nahi thi, non-interactive CI hang hoti.	Variables me explicit default values (<code>sumit-8125</code> , <code>GITOPS-PROJECT-45</code>) set kiye.
5	Terraform state local machine par lost hone ka khata	CI/CD me state ephemeral runner ke destroy hone ke sath delete ho jaati.	S3 Bucket gitops-project-45-bucket create karke <code>versions.tf</code> me remote state backend lagaya.
6	GitHub Actions me <code>Unrecognized named-value: 'secrets'</code>	GitHub Actions step level <code>if:</code> condition me direct <code>secrets.*</code> expression allow nahi karta.	Invalid if condition hata kar direct clean standard <code>aws-actions/configure-aws-credentials</code> step lagaya.
7	Third-party OIDC Debugger crash (<code>apk add go=... exit code 2</code>)	<code>github/actions-oidc-debugger</code> ka Alpine Linux Docker image package update ki wajah se build fail ho raha tha.	Usko hata kar native 2-line Bash/Python inspector lagaya jo 0.1s me JWT payload decode karta hai.
8	<code>Error: Not authorized to perform sts:AssumeRoleWithWebIdentity (403)</code>	GitHub ke naye security update ne token <code>sub</code> me User ID (<code>@294702030</code>) aur Repo ID (<code>@1332272611</code>) add kar diya tha. Trust policy me <code>repo:sumit-8125/*</code> match nahi ho raha tha.	Trust policy me <code>StringLike</code> ke sath <code>repo:sumit-8125@294702030/GITOPS-PROJECT-45@1332272611:*</code> aur <code>repo:sumit-8125*:*</code> daala → OIDC 100% GREEN TICK!
9	Duplicate OIDC Provider crash ka risk	AWS me ek URL ka OIDC provider sirf ek baar ban sakta hai. Console me ban chuka tha, Terraform fir se banata toh <code>EntityAlreadyExists</code> crash hota.	<code>modules/iam/main.tf</code> se redundant resource hata kar direct existing provider ka ARN (<code>local.github_oidc_arn</code>) link kiya.
10	Terraform Format Check step par workflow band hona	<code>terraform fmt -check</code> minor file indentation/spacing difference par exit code 3 dekar pipeline fail kar deta tha.	Workflow me <code>continue-on-error: true</code> lagaya taaki minor formatting issue se deployment na ruke.
11	<code>Unsupported argument: cluster_enabled_log_types</code> aur <code>cluster_addons</code>	Official <code>terraform-aws-modules/eks/aws</code> v21 release ne <code>variables</code> ke aage se <code>cluster_</code> prefix hata diya tha.	<code>modules/eks/main.tf</code> me v21 specification ke according <code>enabled_log_types</code> aur <code>addons</code> me rename kiya.

#	Pehle Kya Dikkat Thi (Issue)	Asli Wajah (Root Cause)	Ab Kya Kiya (Solution)
12	Apply vs Destroy safety control	Testing ke baad resources easily delete karne ka option chahiye tha taaki AWS bills na bane.	Workflow dispatch me input dropdown banaya jisse user apply ya destroy choose kar sake.

4. Operational Runbook (Kaise Run aur Destroy Karein)

▶ Deploy Infrastructure

1. Open GitHub Actions → **01 - Terraform CI/CD Pipeline**.
2. Click **Run workflow**.
3. Dropdown me **apply** select karein aur Green button daba dein.
4. EKS, VPC, RDS, ECR ~12-15 minutes me live ho jayenge.

◻ Destroy Infrastructure (Stop Billing)

1. Open GitHub Actions → **01 - Terraform CI/CD Pipeline**.
2. Click **Run workflow**.
3. Dropdown me **destroy** select karein aur Green button daba dein.
4. Saare costly resources (EKS control plane, NAT gateway, RDS) automatically clean ho jayenge.